Experiment 5 – Wireshark Packet Capture and Analysis

1. Aim

To use Wireshark to capture network traffic and examine packets by applying protocol filters such as TCP, UDP, ARP, DNS, HTTP, IP, ICMP, and DHCP.

2. Description

Wireshark is a packet-capturing and network troubleshooting application that records traffic in real time and presents protocol information in an understandable format. It helps users examine network activity, inspect protocol fields, apply display filters, and investigate communication issues.

Packets transmitted and received by the computer can be examined in Wireshark using the Packet List, Packet Details, and Packet Bytes panels.

3. Packet Capture

The appropriate network interface was chosen in Wireshark. The capture was configured to stop after 100 packets, and the captured data was saved for later inspection.

Display filters were used to view packets belonging to the following protocols:

- TCP/UDP
- ARP
- DNS
- HTTP
- IP/ICMP
- DHCP

Flow graphs were also created to examine communication patterns involving TCP/UDP and DNS packets.

4. Observation

Wireshark was used to observe and study network traffic exchanged between devices. When promiscuous mode is supported and enabled, a network interface may capture traffic passing on the network beyond packets specifically addressed to that computer; the actual traffic visible depends on the network and interface.

ARP does not use TCP or UDP transport headers. It helps determine the hardware (MAC) address associated with an IPv4 address.

DNS commonly uses UDP port 53, although TCP can be used in certain cases.

HTTP traditionally uses TCP port 80 by default.

A broadcast IP address delivers traffic to all hosts on a particular subnet. For example, for 192.168.1.0/24, the broadcast address is 192.168.1.255.

5. Outcome

Wireshark was used to capture and inspect network packets. Filters were applied to view different protocol types, and packet communication flows were examined with the available analysis features.

6. Challenges Faced

Choosing the correct network interface before starting the capture.

Finding the relevant protocol packets within the captured traffic.

Entering suitable filters for TCP, UDP, ARP, DNS, HTTP, IP/ICMP, and DHCP.

Interpreting the fields and information shown in packet details.

Creating and understanding the flow graph.

Capturing the required number of packets for analysis.

7. Result

Thus, network traffic was captured and examined successfully using Wireshark. Protocol filters were applied, and the selected packets were inspected and analysed.